

CuteDNS: DNS Resolver using AF_XDP Sockets

Accelerating DNS resolution through zero-copy packet processing



CS331 - Computer Networks

Professor - Sameer G. Kulkarni

TA - Naveen Tiwari

Group No: 21 | Project No. 31

Aarsh Wankar (23110003) Team Leader

Abhinav Khot (23110006)

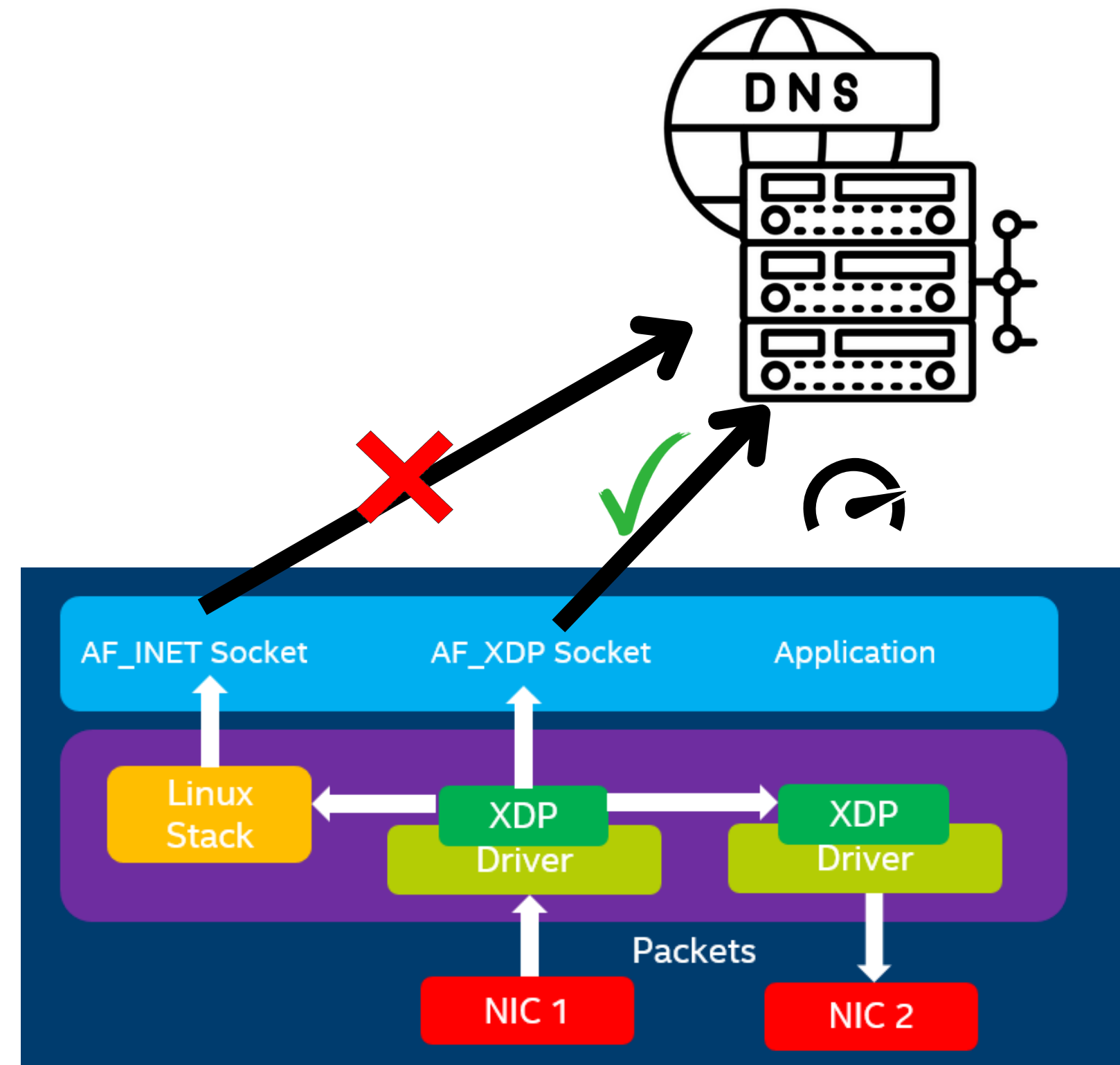
Jaskirat Singh Maskeen (23110146)

Karan Sagar Gandhi (23110157)

Indian Institute of Technology Gandhinagar
Palaj, Gujarat - 382355

Equal contributions, ordered based on roll number.

Source: <https://github.com/Karan-Gandhi/CS331-Computer-Networks-Project>



Problem Statement

Implement a DNS resolver that sends and receives DNS packets using an AF_XDP (XSK) socket instead of the usual UDP/TCP socket.

Questions we addressed in this project:

1. What is XDP?
2. How to set up a test bench that supports zero-copy?
3. How to multithread the resolver?
4. Is AF_XDP better than the standard UDP socket (AF_INET)?
5. How does cache affect the performance? (Cache policies?)

Our Workflow:

1. Literature Review
2. Making an XDP-supported DNS filter.
3. Porting our custom resolver from Assignment 2 to C (Including cache).
4. Making two userspace applications to receive packets, one from AF_XDP and the other from AF_INET, and resolve DNS.
5. Benchmark and compare performance.

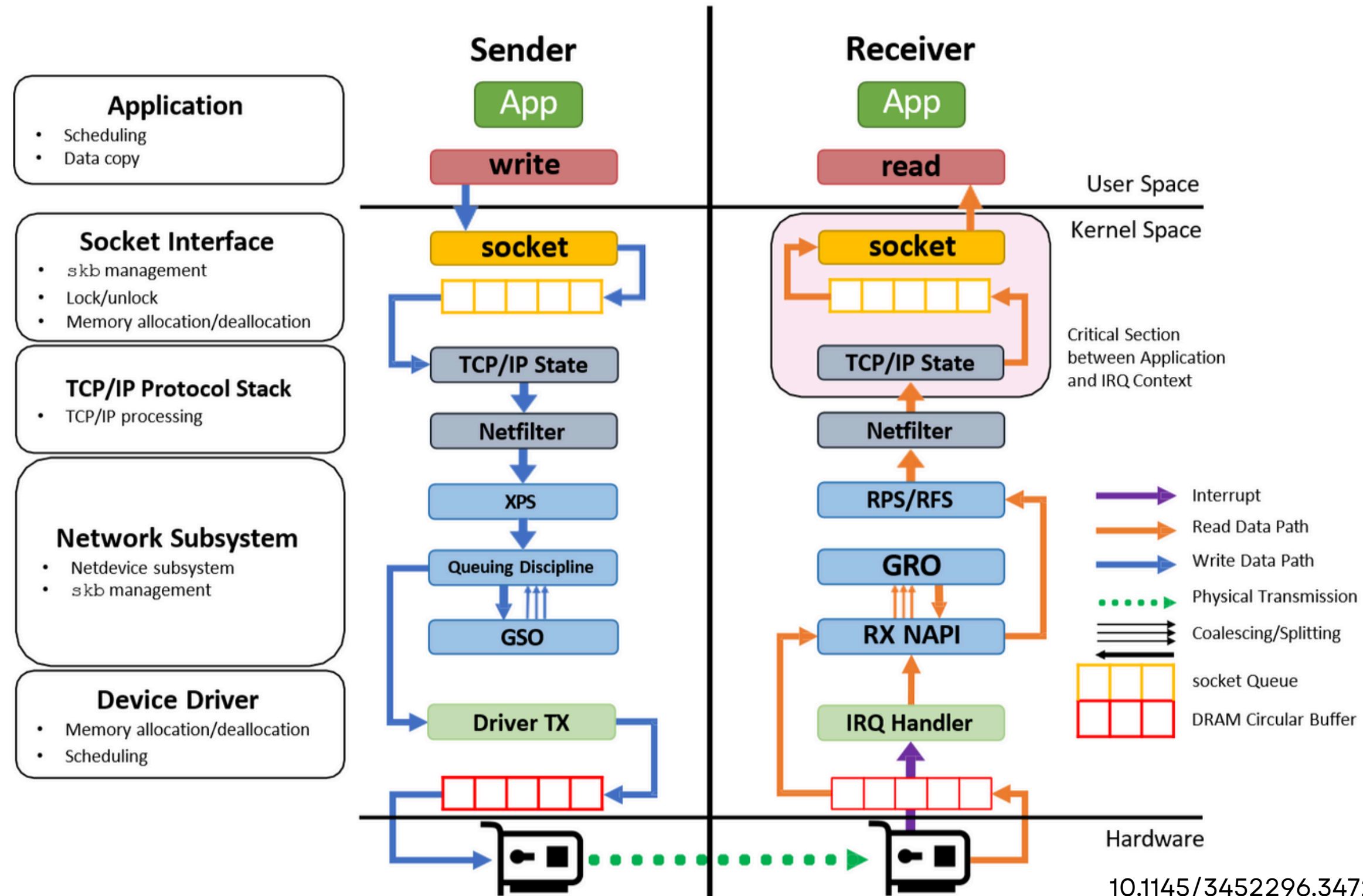
Why is the Linux Network Stack "Slow"?

Why is the Linux Network Stack "Slow"?

- A packet in Linux passes through many tightly coupled subsystems before it leaves or reaches user space.

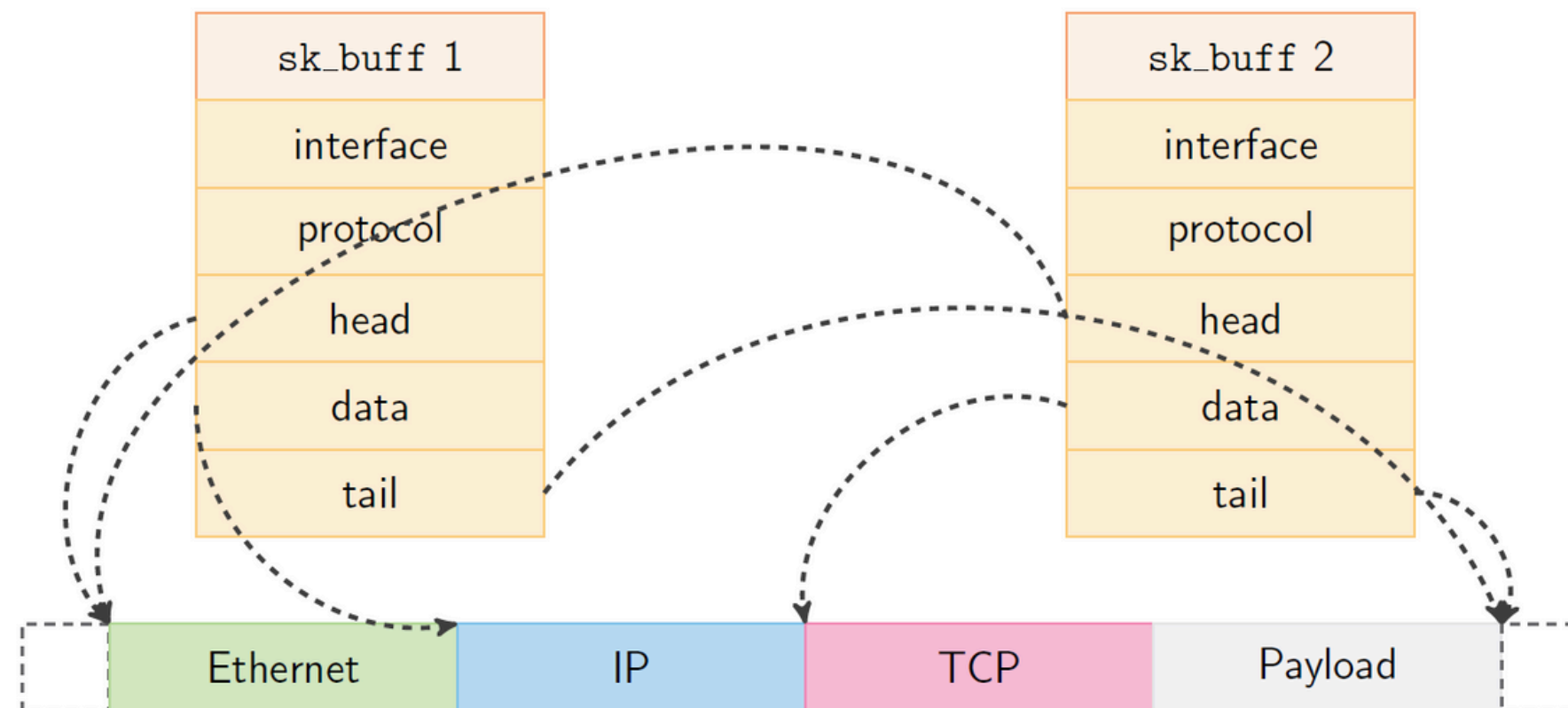
Why is the Linux Network Stack "Slow"?

- A packet in Linux passes through many tightly coupled subsystems before it leaves or reaches user space.



Why is the Linux Network Stack "Slow"?

- A packet in Linux passes through many tightly coupled subsystems before it leaves or reaches user space.
- The main bottleneck - **sk_buff** (stores header pointers, metadata, etc.)
- Why? Because each allocation/deallocation and clone of a sk_buff involves:
 - Dynamic memory allocation
 - Atomic reference counting
 - Cache misses due to its large size (~320 bytes per skb)

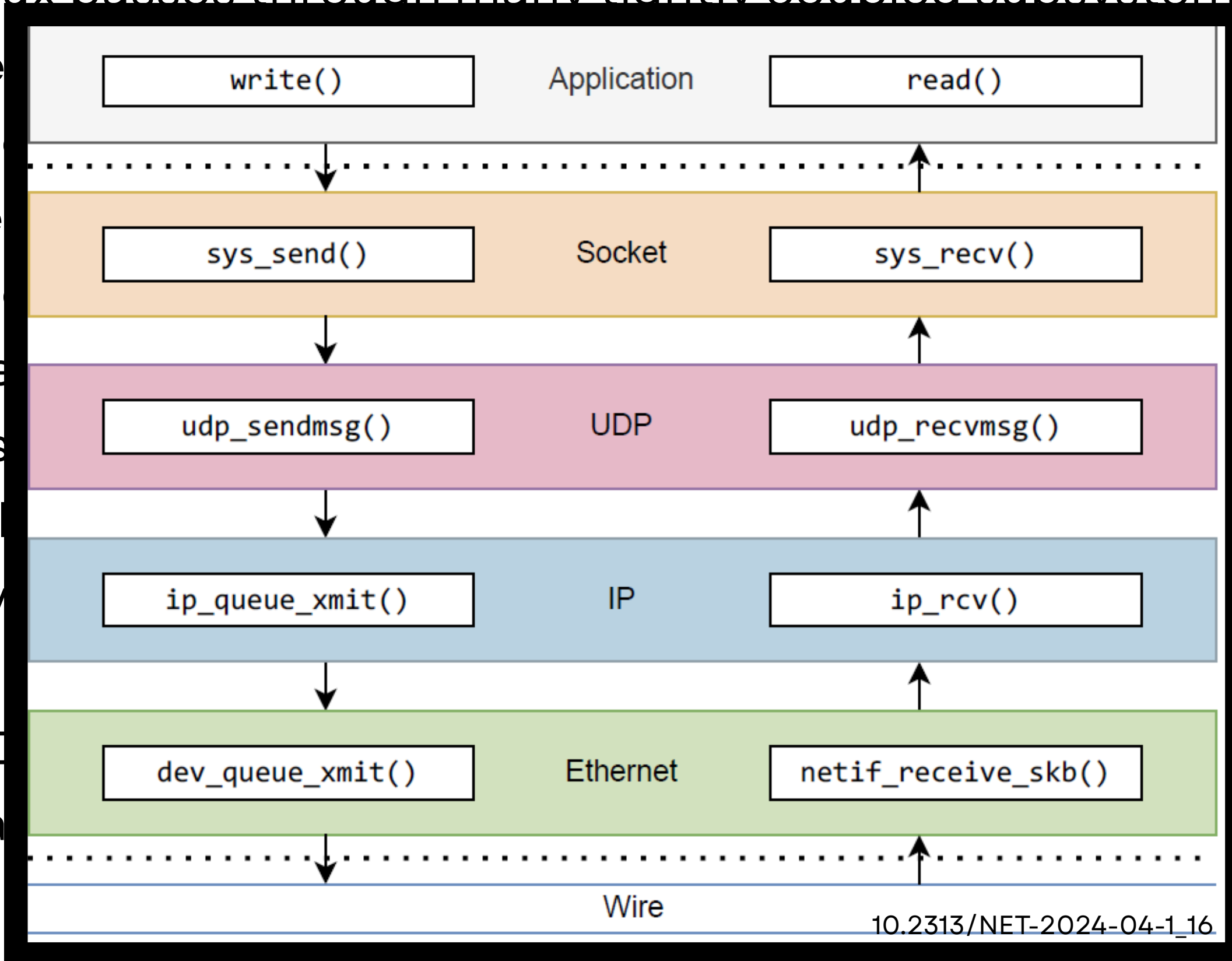


Why is the Linux Network Stack "Slow"?

- A packet in Linux passes through many tightly coupled subsystems before it leaves or reaches user space.
- The main bottleneck - **sk_buff** (stores header pointers, metadata, etc.)
- Why? Because each allocation/deallocation and clone of a sk_buff involves:
 - Dynamic memory allocation
 - Atomic reference counting
 - Cache misses due to its large size (~320 bytes per skb)
- “For 100 Gbps links, data copy from kernel to application buffers accounts for >50% of total CPU cycles at the receiver.” - 10.1145/3452296.3472888
- Each received packet must be:
 - Copied by DMA from NIC → kernel buffer (DRAM or L3 Cache)
 - Copied again from kernel → user space buffer.
- Note: Even though sk_buff has pointers, the payload has to be copied because of memory protection between kernel and user space

Why is the Linux Network Stack "Slow"?

- A packet in Linux passes through many tightly coupled subsystems before it leaves or reaches user space (e.g., applications, etc.)
- The main bottleneck involves:
- Why? Because
 - Dynamic memory allocation
 - Atomic reference counting
 - Cache misses
- “For 100 Gbps line rates, this accounts for >50% of total CPU cycles”
- Each received packet involves:
 - Copied by DMA
 - Copied again
- Note: Even though this is slowed because of memory protection between kernel and user space



Bypassing the Kernel with eBPF & XDP

Bypassing the Kernel with eBPF & XDP

- eBPF (Extended Berkeley Packet Filter):
 - A safe, high-performance virtual machine inside the Linux kernel.
 - "attach" small programs to hooks at runtime to modify kernel behavior.

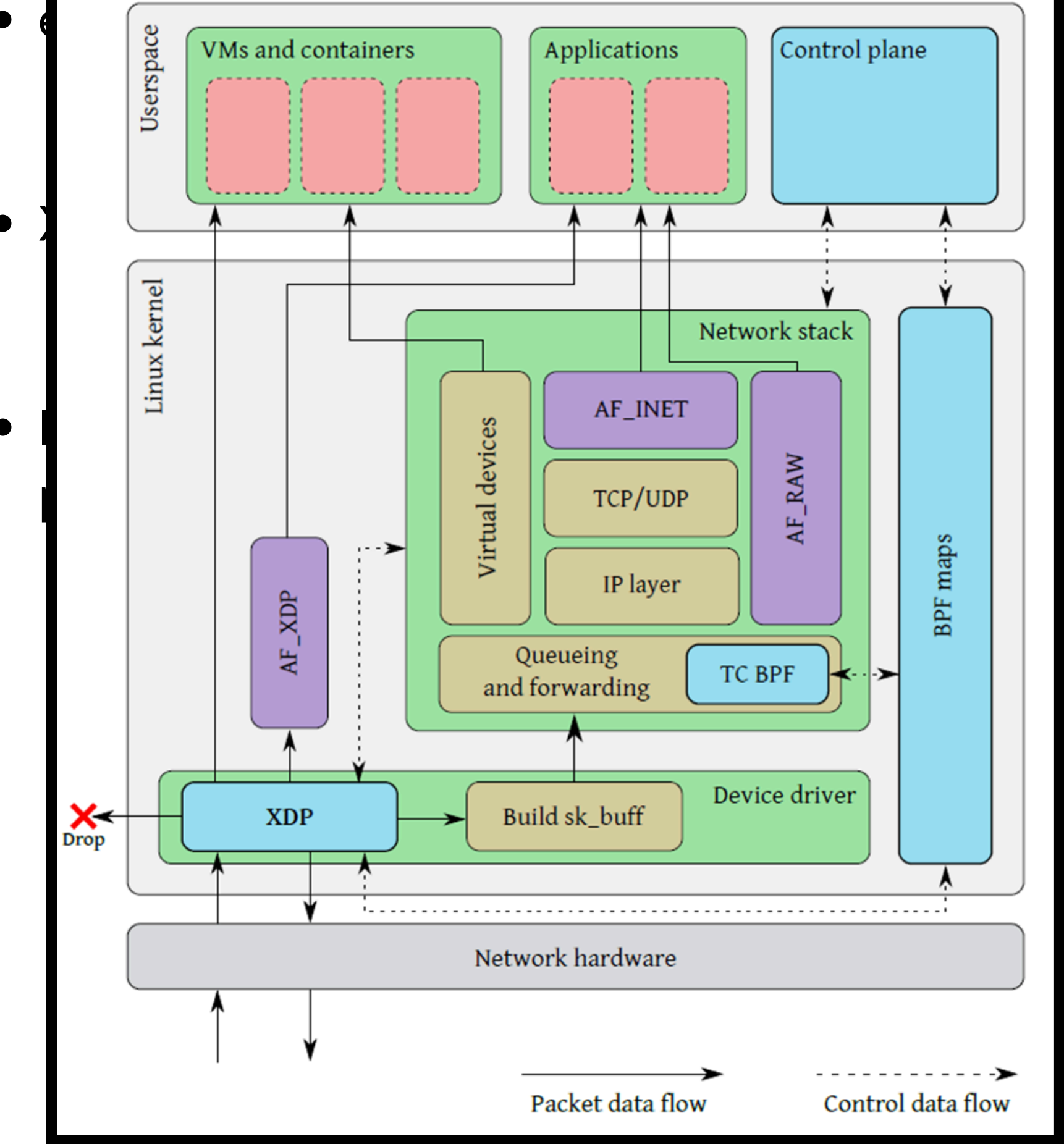
Bypassing the Kernel with eBPF & XDP

- eBPF (Extended Berkeley Packet Filter):
 - A safe, high-performance virtual machine inside the Linux kernel.
 - "attach" small programs to hooks at runtime to modify kernel behavior.
- XDP (eXpress Data Path):
 - The earliest possible hook, running directly in the NIC driver.
 - It sees the packet before the kernel allocates any memory for it (**no sk_buff**).

Bypassing the Kernel with eBPF & XDP

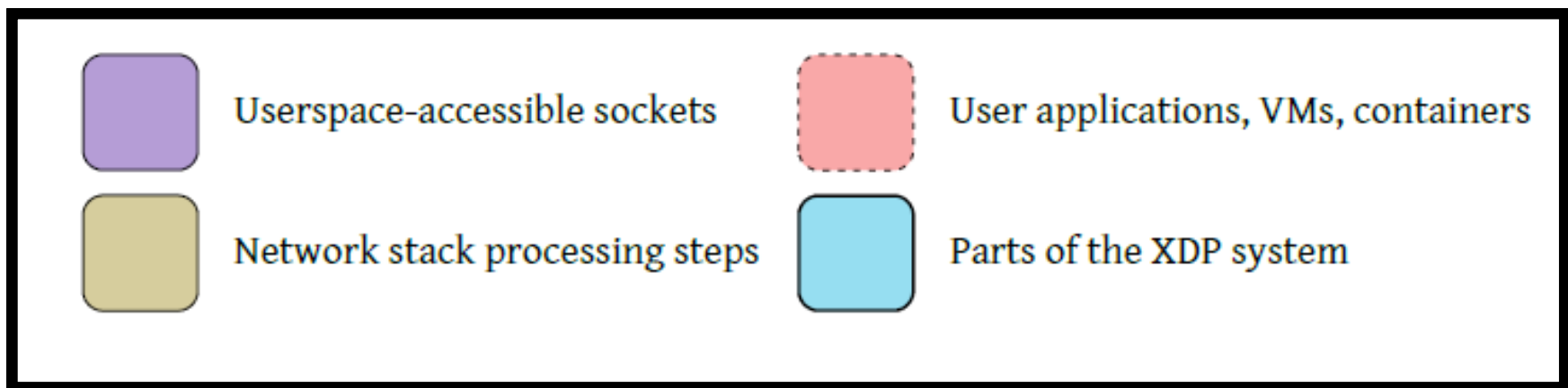
- eBPF (Extended Berkeley Packet Filter):
 - A safe, high-performance virtual machine inside the Linux kernel.
 - "attach" small programs to hooks at runtime to modify kernel behavior.
- XDP (eXpress Data Path):
 - The earliest possible hook, running directly in the NIC driver.
 - It sees the packet before the kernel allocates any memory for it (**no sk_buff**).
- Ideal for ultra-fast processing, like DDoS mitigation or routing, at millions of packets per second.

Bypassing the Kernel with eBPF & XDP



• Machine inside the Linux kernel.
• Runtime to modify kernel behavior.

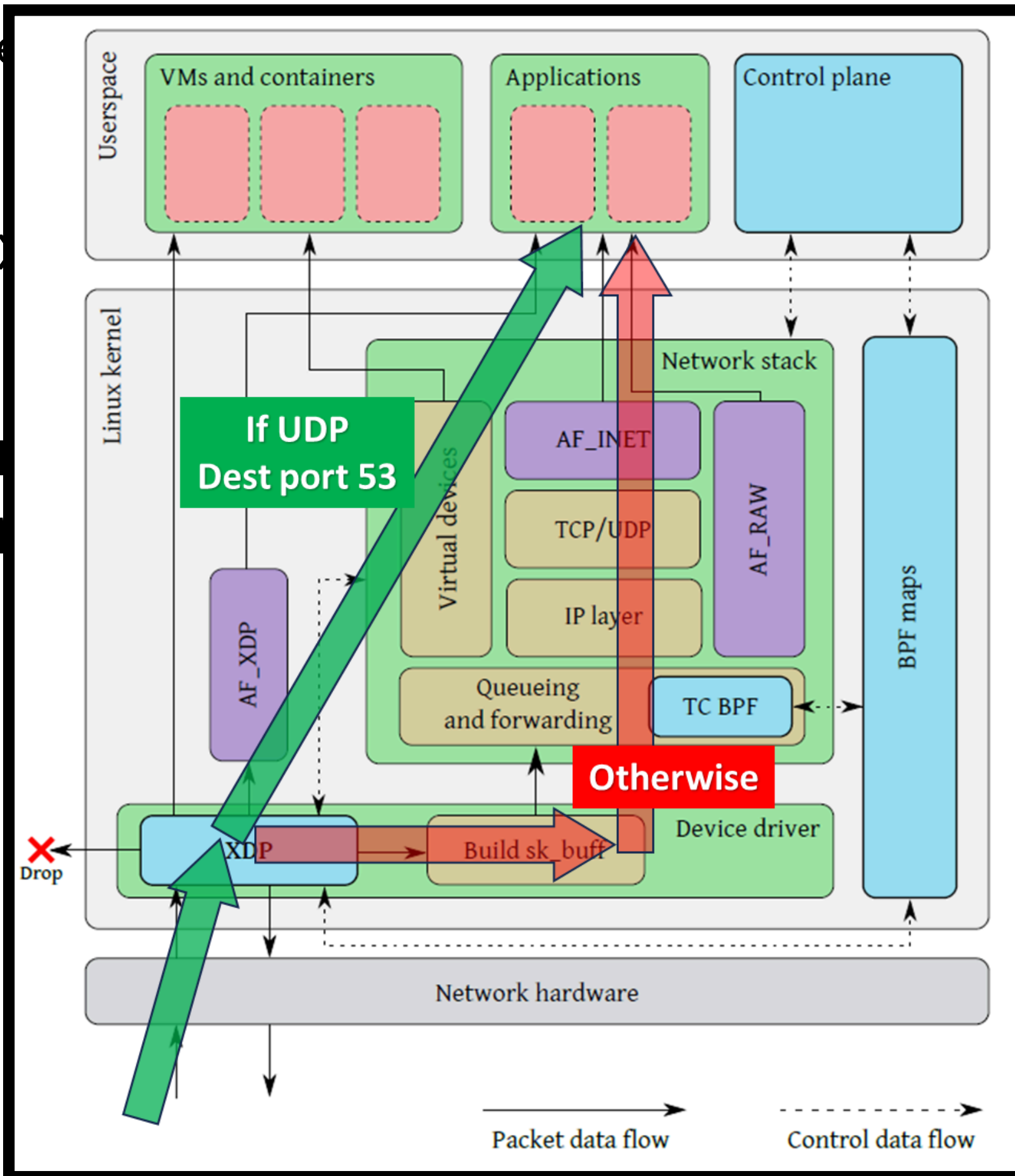
• Directly in the NIC driver.
• Allocates any memory for it (**no sk_buff**).
• Mitigation or routing, at millions of packets



10.1145/3281411.3281443

Bypassing the Kernel with eBPF & XDP

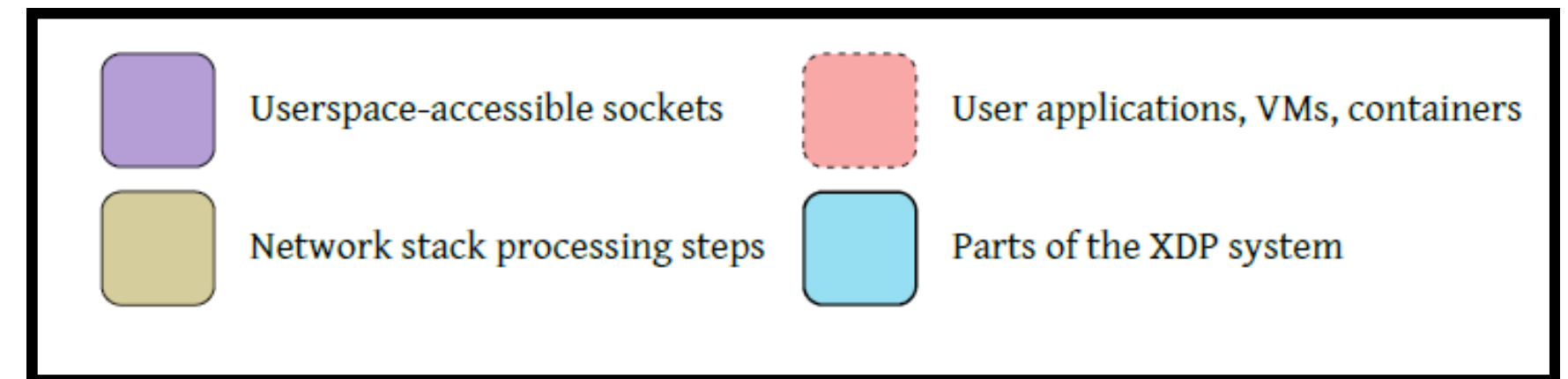
-
-
-
-



machine inside the Linux kernel.
runtime to modify kernel behavior.

directly in the NIC driver.

allocates any memory for it (**no `sk_buff`**).
mitigation or routing, at millions of packets



10.1145/3281411.3281443

Bypassing the Kernel with eBPF & XDP

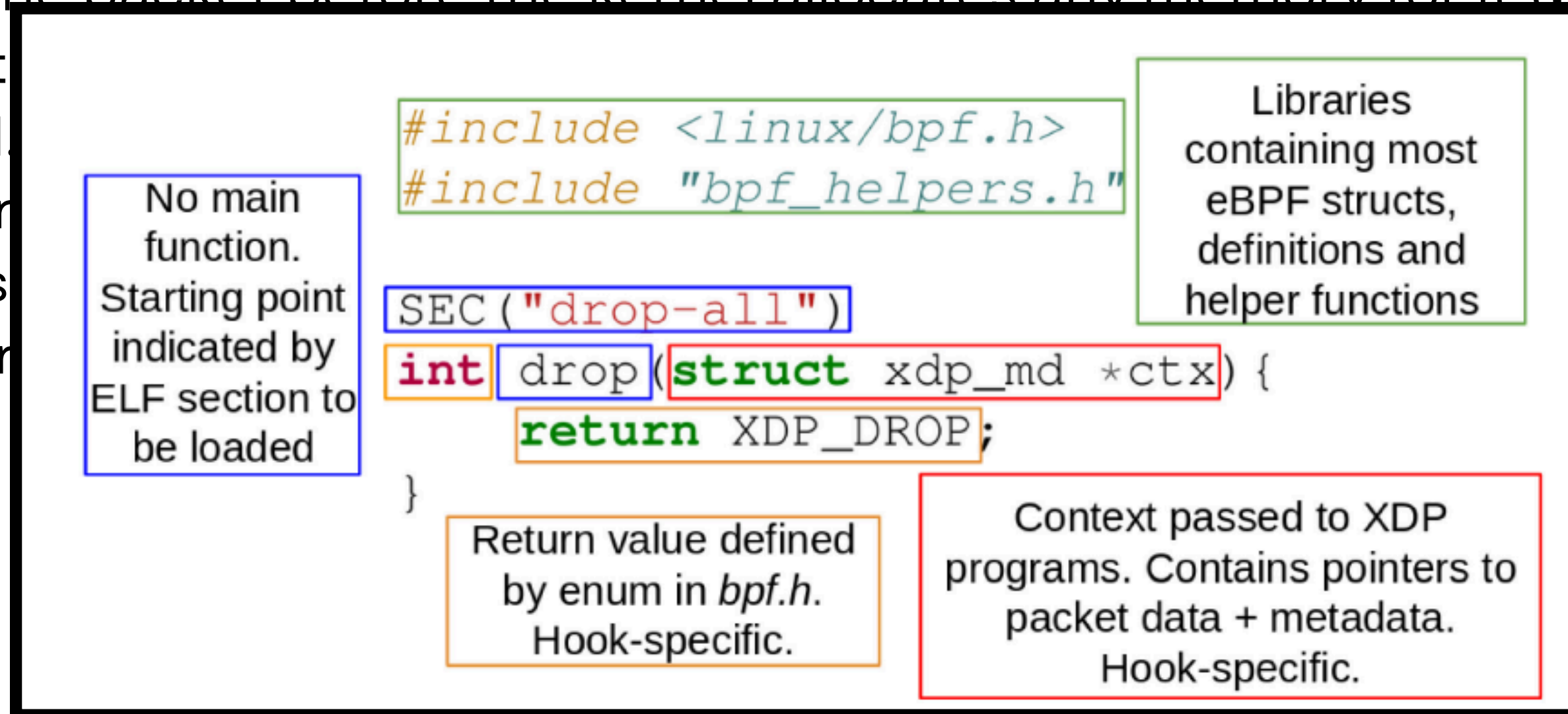
- eBPF (Extended Berkeley Packet Filter):
 - A safe, high-performance virtual machine inside the Linux kernel.
 - "attach" small programs to hooks at runtime to modify kernel behavior.
- XDP (eXpress Data Path):
 - The earliest possible hook, running directly in the NIC driver.
 - It sees the packet before the kernel allocates any memory for it (**no sk_buff**).
- Ideal for ultra-fast processing, like DDoS mitigation or routing, at millions of packets per second.
- XDP uses an eBPF program to inspect/drop/modify/redirect packets before allocating socket buffers (skb).

Bypassing the Kernel with eBPF & XDP

- eBPF (Extended Berkeley Packet Filter):
 - A safe, high-performance virtual machine inside the Linux kernel.
 - "attach" small programs to hooks at runtime to modify kernel behavior.
- XDP (eXpress Data Path):
 - The earliest possible hook, running directly in the NIC driver.
 - It sees the packet before the kernel allocates any memory for it (**no sk_buff**).
- Ideal for ultra-fast processing, like DDoS mitigation or routing, at millions of packets per second.
- XDP uses an eBPF program to inspect/drop/modify/redirect packets before allocating socket buffers (skb).
- The eBPF program can communicate with the userspace using BPF maps.

Bypassing the Kernel with eBPF & XDP

- eBPF (Extended Berkeley Packet Filter):
 - A safe, high-performance virtual machine inside the Linux kernel.
 - "attach" small programs to hooks at runtime to modify kernel behavior.
- XDP (eXpress Data Path):
 - The earliest possible hook, running directly in the NIC driver.
 - It sees the packet before the kernel allocates any memory for it (**no sk_buff**).
- Ideal for ultra-high-speed processing of packets.
- XDP uses an alternative memory allocation strategy.
- The eBPF program is loaded into the kernel before it can be attached to a hook.



10.1145/3371038

Structure of an eBPF program. Here it drops all the packets.

How to talk to userspace ?

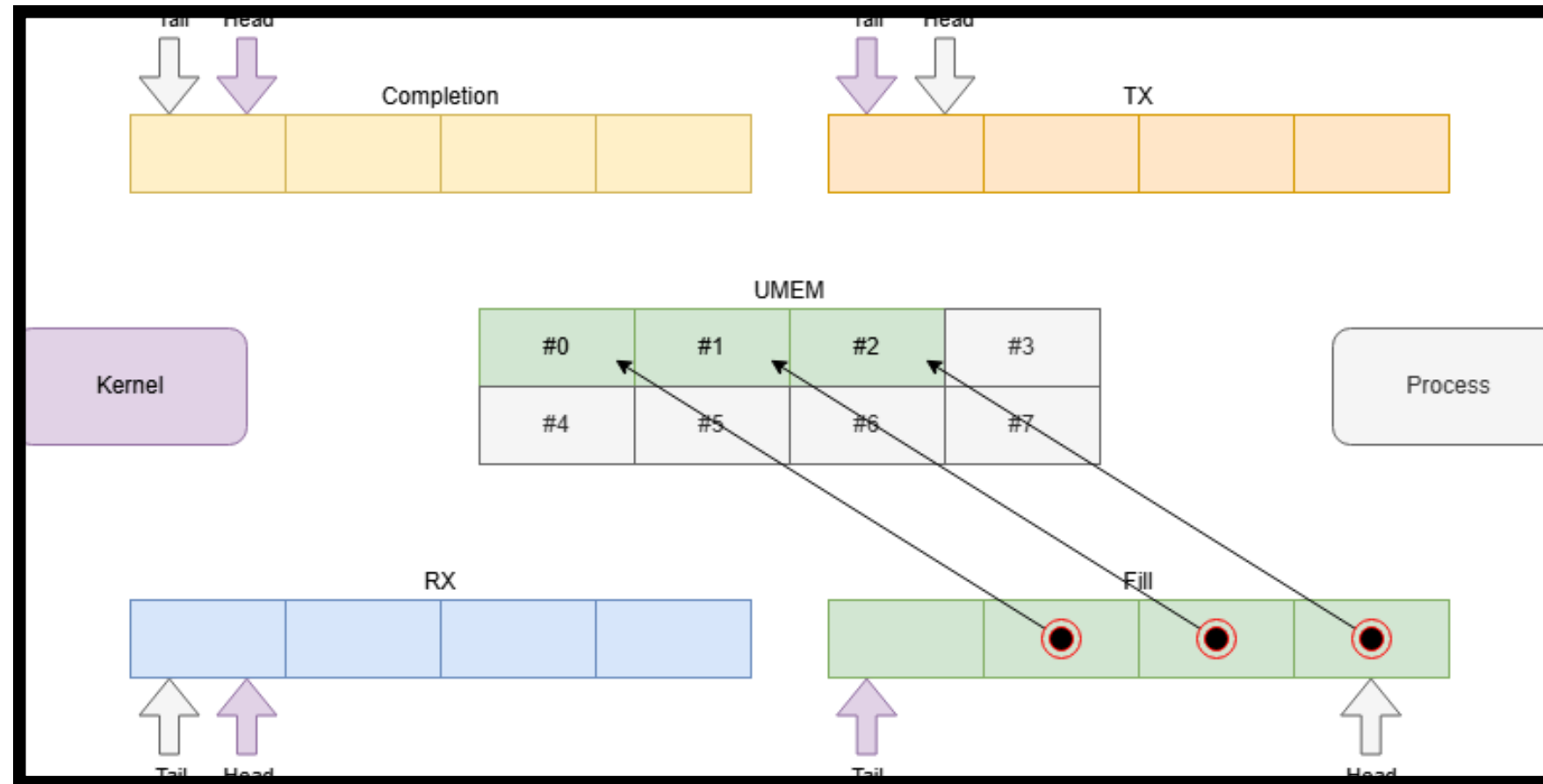
How to talk to userspace ?

- Four Rings
 - **FILL Ring**: Userspace provides buffers (addresses) to the kernel which fills the addresses to the respective Rx ring of the NIC.
 - **RX Ring**: Kernel gives packet descriptors to userspace after fill.
 - **TX Ring**: Userspace gives packet descriptors to the kernel when sending.
 - **Completion Ring**: Kernel returns descriptors to the user after send is done.

How to talk to userspace ?

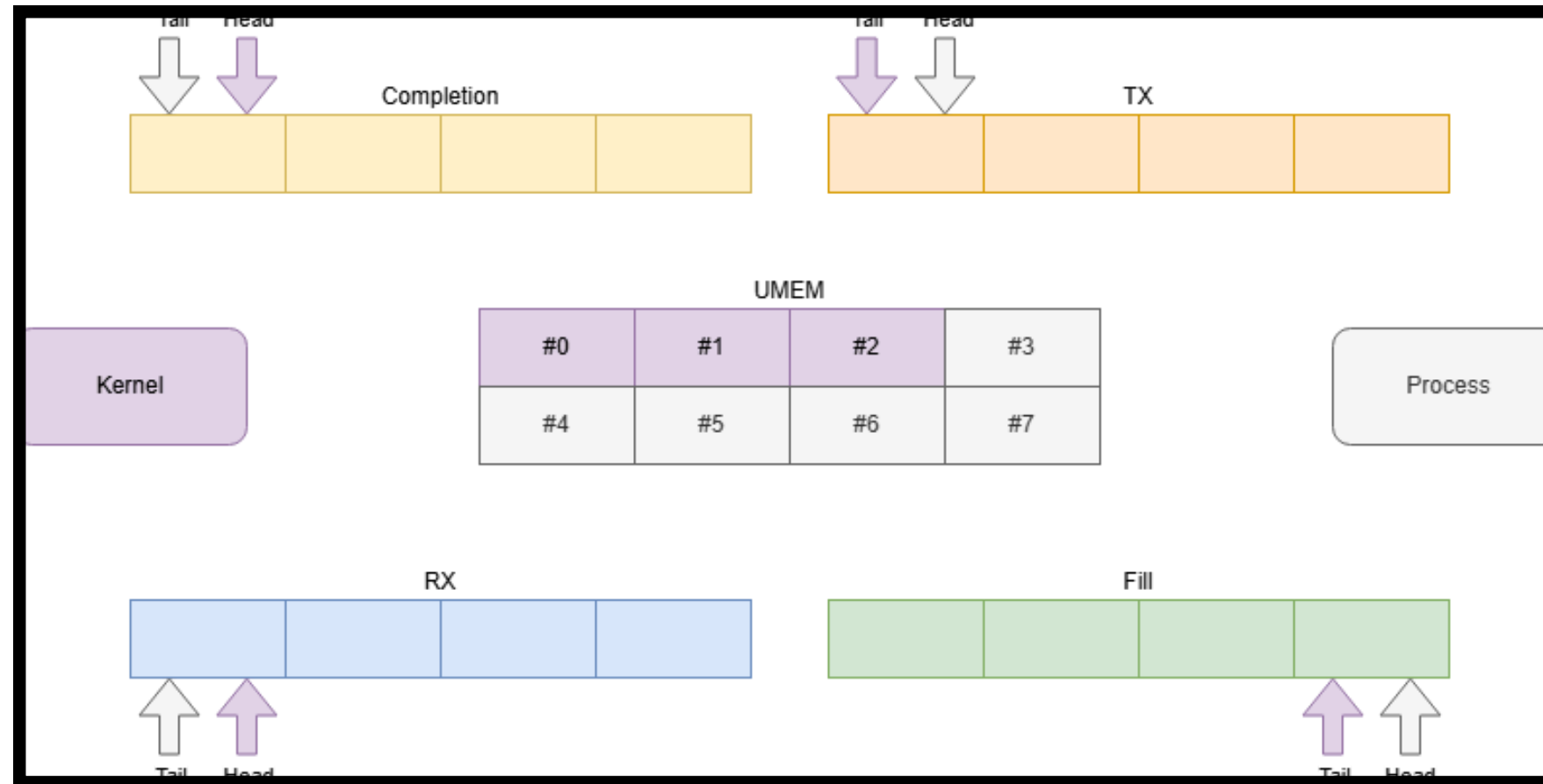
- Four Rings
 - **FILL Ring**: Userspace provides buffers (addresses) to the kernel which fills the addresses to the respective Rx ring of the NIC.
 - **RX Ring**: Kernel gives packet descriptors to userspace after fill.
 - **TX Ring**: Userspace gives packet descriptors to the kernel when sending.
 - **Completion Ring**: Kernel returns descriptors to the user after send is done.
- Ingress:
 - a. **UMEM**: The application allocates a contiguous UMEM region, registers it with the kernel, by filling the FILL ring and maps it for direct packet access.

How to talk to userspace ?



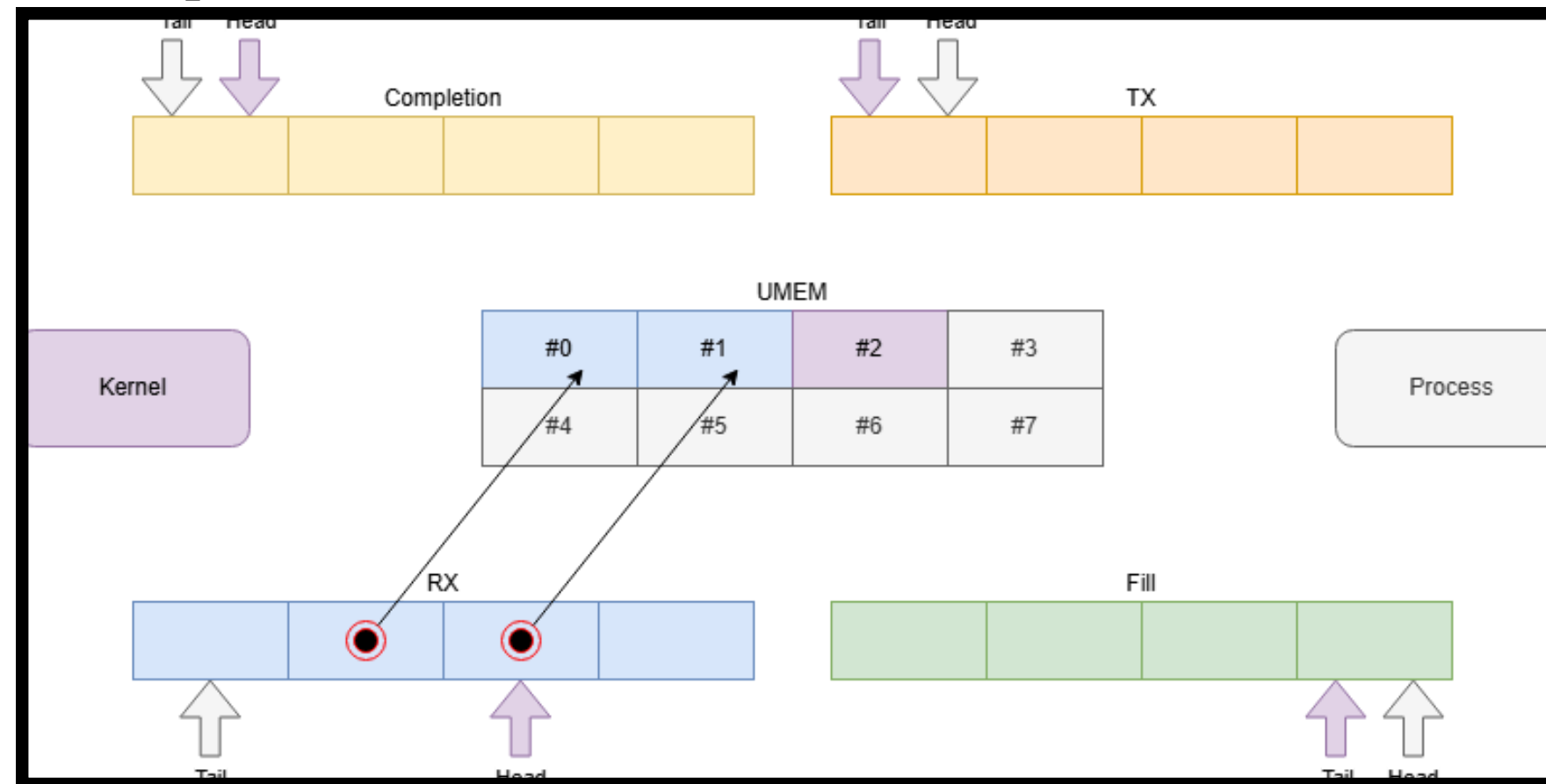
b. **FILL ring**: The application posts buffer descriptors (UMEM offsets) into the FILL ring, informing the NIC where to store incoming packets.

How to talk to userspace ?



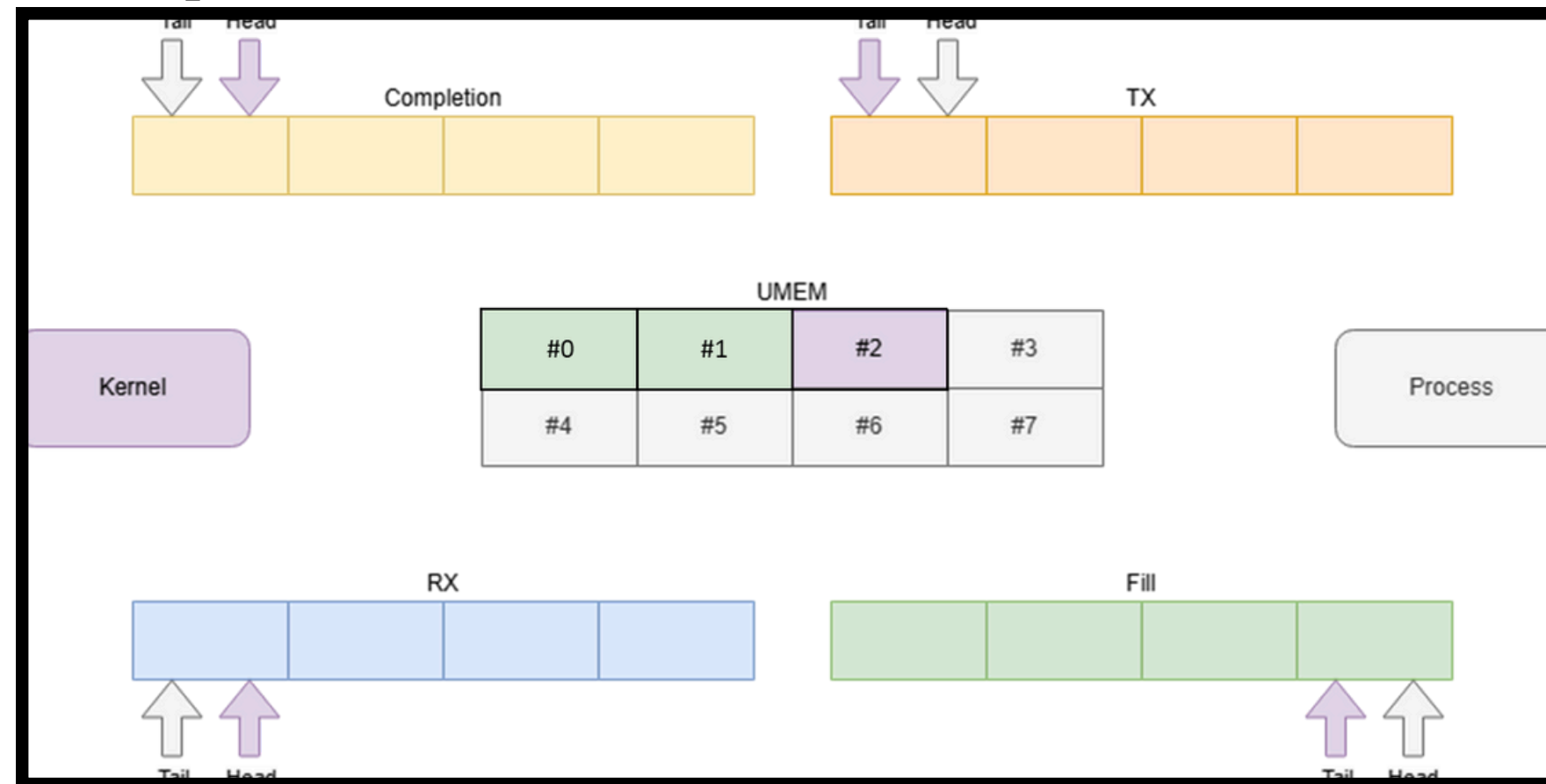
- b. **FILL ring:** The application posts buffer descriptors (UMEM offsets) into the FILL ring, informing the NIC where to store incoming packets.
- c. **Packet arrival:** The NIC receives a packet and, via the UMEM address present in the RX rings of the NIC, using DMA, writes it directly into a UMEM buffer without copying.

How to talk to userspace ?



- b. **FILL ring:** The application posts buffer descriptors (UMEM offsets) into the FILL ring, informing the NIC where to store incoming packets.
- c. **Packet arrival:** The NIC receives a packet and, via the UMEM address present in the RX rings of the NIC, using DMA, writes it directly into a UMEM buffer without copying.
- d. **RX ring:** The kernel places a descriptor (address, length, flags) into the RX ring, transferring buffer ownership to user space.

How to talk to userspace ?



- b. **FILL ring**: The application posts buffer descriptors (UMEM offsets) into the FILL ring, informing the NIC where to store incoming packets.
- c. **Packet arrival**: The NIC receives a packet and, via the UMEM address present in the RX rings of the NIC, using DMA, writes it directly into a UMEM buffer without copying.
- d. **RX ring**: The kernel places a descriptor (address, length, flags) into the RX ring, transferring buffer ownership to user space.
- e. **Consume & reuse**: The application reads packets from the RX ring, processes them, and returns buffers to the FILL or TX ring for reuse.

BPF Maps and Our DNS Resolver

- The map tells the eBPF program which userspace socket to redirect the incoming packet.

```
struct {  
    __uint(type, BPF_MAP_TYPE_XSKMAP);  
    __uint(max_entries, 64);  
    __type(key, __u32);  
    __type(value, __u32);  
} xsk_map SEC(".maps");
```

BPF Maps and Our DNS Resolver

- The map tells the eBPF program which userspace socket to redirect the incoming packet.
- The keys are queue indices (u32) and the values are XSK file descriptors (u32) or references to the socket.

```
struct {  
    __uint(type, BPF_MAP_TYPE_XSKMAP);  
    __uint(max_entries, 64);  
    __type(key, __u32);  
    __type(value, __u32);  
} xsks_map SEC(".maps");
```

BPF Maps and Our DNS Resolver

- The map tells the eBPF program which userspace socket to redirect the incoming packet.
- The keys are queue indices (u32) and the values are XSK file descriptors (u32) or references to the socket.
- eBPF uses a helper, **bpf_redirect_map()**, with the XSKMAP and a key to redirect the packet into AF_XDP socket corresponding to that key.

```
struct {  
    __uint(type, BPF_MAP_TYPE_XSKMAP);  
    __uint(max_entries, 64);  
    __type(key, __u32);  
    __type(value, __u32);  
} xsk_map SEC(".maps");
```

```
SEC("xdp")  
int xdp_pass_prog(struct xdp_md *ctx)  
{  
    void *data_end = (void *)(long)ctx->data_end;  
    void *data = (void *)(long)ctx->data;  
    struct ethhdr *eth = data;  
    if (((void *)eth + 1) > data_end) ||  
        (eth->h_proto != __constant_htons(ETH_P_IP)))  
        return XDP_PASS;  
    struct iphdr *ip = (void *)eth + 1;  
    if (((void *)ip + 1) > data_end) ||  
        (ip->protocol != IPPROTO_UDP))  
        return XDP_PASS;  
    struct udphdr *udp = (void *)ip + (ip->ihl * 4);  
    if ((void *)udp + 1 > data_end)  
        return XDP_PASS;  
  
    __u32 index = ctx->rx_queue_index;  
  
    if (udp->dest == __constant_htons(53)) {  
        return bpf_redirect_map(&xsk_map, index, XDP_ABORTED);  
    }  
    return XDP_PASS;  
}
```

BPF Maps and Our DNS Resolver

- The map tells the eBPF program which userspace socket to redirect the incoming packet.
- The keys are queue indices (u32) and the values are XSK file descriptors (u32) or references to the socket.
- eBPF uses a helper, **bpf_redirect_map()**, with the XSKMAP and a key to redirect the packet into AF_XDP socket corresponding to that key.
- In the user space, the application tied to that socket polls the RX ring, sees a descriptor, reads the packet from the UMEM memory, and processes it.

```
struct {  
    __uint(type, BPF_MAP_TYPE_XSKMAP);  
    __uint(max_entries, 64);  
    __type(key, __u32);  
    __type(value, __u32);  
} xsks_map SEC(".maps");
```

BPF Maps and Our DNS Resolver

- The map tells the eBPF program which userspace socket to redirect the incoming packet.
- The keys are queue indices (u32) and the values are XSK file descriptors (u32) or references to the socket.
- eBPF uses a helper, **bpf_redirect_map()**, with the XSKMAP and a key to redirect the packet into AF_XDP socket corresponding to that key.
- In the user space, the application tied to that socket polls the RX ring, sees a descriptor, reads the packet from the UMEM memory, and processes it.

```
struct {
    __uint(type, BPF_MAP_TYPE_XSKMAP);
    __uint(max_entries, 64);
    __type(key, __u32);
    __type(value, __u32);
} xsk_map SEC(".maps");
```

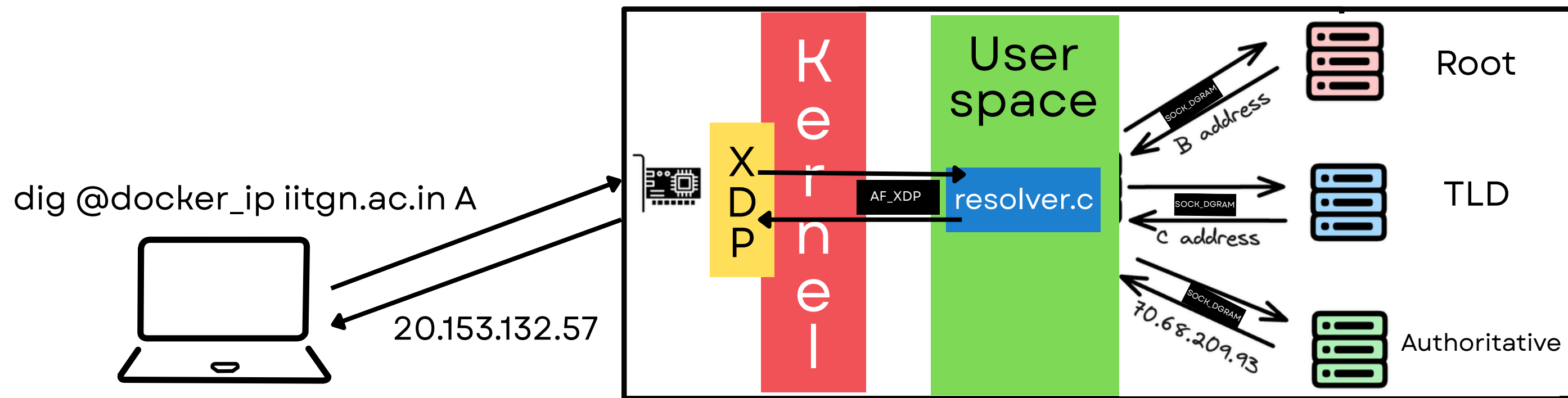
```
static void rx_and_process(struct config *cfg,
                           struct xsk_socket_info *xsk_socket)
{
    struct pollfd fds[2];
    int ret, nfds = 1;

    memset(fds, 0, sizeof(fds));
    fds[0].fd = xsk_socket__fd(xsk_socket->xsk);
    fds[0].events = POLLIN;

    while(!global_exit) {
        if (cfg->xsk_poll_mode) {
            ret = poll(fds, nfds, -1);
            if (ret <= 0 || ret > 1)
                continue;
        }
        handle_receive_packets(xsk_socket);
    }
}
```

BPF Maps and Our DNS Resolver

- The map tells the eBPF program which userspace socket to redirect the incoming packet.
- The keys are queue indices (u32) and the values are XSK file descriptors (u32) or references to the socket.
- eBPF uses a helper, **bpf_redirect_map()**, with the XSKMAP and a key to redirect the packet into AF_XDP socket corresponding to that key.
- In the user space, the application tied to that socket polls the RX ring, sees a descriptor, reads the packet from the UMEM memory, and processes it.



Benchmarking and Results

- We test the following metrics: latency, throughput (in terms of queries per second), and CPU overhead.
- We use **DNSPerf** to simulate sending requests from three clients in parallel, each with a 15-second timeout. The test is run for 60 seconds from the host.

Config	Avg. Latency (s)	Std.Deviation (s)	Throughput (q/s)
AF_XDP-LRU	0.758775	1.072431	2.450
Normal-LRU	1.36117	2.215635	2.944
AFXDP-Hashcace	0.916267	1.639238	2.463
Normal-HashCace	1.540820	2.807204	3.040

The XDP counterparts outperform the normal ones by a large margin in terms of latency (both avg and std.)

Throughput is lower because we use only one XDP socket for egress

Benchmarking and Results

Configuration	CPU Cycles/query	Instructions/cy cle
AF_XDP-LRU	1829269090	1.98
Normal-LRU	1518779	0.26
AF_XDP- HashCache	1783024441	1.99
Normal- HashCache	1918129	0.23

- **Although the number of cycles is much higher for the XDP versions, the instructions per cycles is much better.**
- **This indicates a more efficient data processing pathway for the packets which does not involve much indirection and overhead.**



Thankyou and Quesitons

Aarsh Wankar (23110003)

Abhinav Khot (23110006)

Jaskirat Singh Maskeen (23110146)

Karan Sagar Gandhi (23110157)

References

1. Bardinelli, J., Zhang, Y., Su, J., Huang, L., Parilla, A., Jarvi, R., Kulkarni, S. G., & Zhang, W. (2024). **HyDNS: Acceleration of DNS through Kernel Space Resolution**. Proceedings of the SIGCOMM Workshop on eBPF and Kernel Extensions, 58–64. <https://doi.org/10.1145/3672197.3673439>
2. Cai, Q., Chaudhary, S., Vuppalapati, M., Hwang, J., & Agarwal, R. (2021). **Understanding host network stack overheads**. Proceedings of the 2021 ACM SIGCOMM 2021 Conference, 65–77. <https://doi.org/10.1145/3452296.3472888>
3. Høiland-Jørgensen, T., Brouer, J. D., Borkmann, D., Fastabend, J., Herbert, T., Ahern, D., & Miller, D. (2018). **The Express Data Path**. Proceedings of the 14th International Conference on Emerging Networking EXperiments and Technologies, 54–66. <https://doi.org/10.1145/3281411.3281443>
4. Stephan, A., & Wüstrich, L. (n.d.). **The path of a packet through the Linux kernel**. Retrieved from <https://api.semanticscholar.org/CorpusID:269777344>
5. Vieira, M. A., Castanho, M. S., Pacífico, R. D., Santos, E. R., Júnior, E. P., & Vieira, L. F. (2020). **Fast packet processing with EBPF and XDP**. ACM Computing Surveys, 53(1), 1–36. <https://doi.org/10.1145/3371038>